

Week 05: Neural Network Practical Task

Lab 1: Building an Artificial Neural Network (ANN) from Scratch using Python and NumPy

Objective:

The goal of this lab is to help interns understand the inner workings of Artificial Neural Networks by building a simple feedforward ANN from scratch using only Python and NumPy. By the end of this lab, you should be able to:

- Initialize network parameters
 - Implement activation functions
 - Perform forward and backward propagation
 - Train the model using gradient descent
 - Make predictions on test data
-

Step 1: Import Required Libraries

Task:

Import the essential library used for numerical operations in Python.

Hint: Use the standard alias for NumPy.

Import the required library here

Step 2: Initialize Network Parameters

Task:

Write a function to initialize the weights and biases for a 2-layer neural network (1 hidden layer).

- Use small random numbers for weights (scale appropriately)

- Initialize biases with zeros
- Use a fixed seed for reproducibility

Hint:

- Weights matrix for layer 1 should have shape `(hidden_size, input_size)`
- Weights matrix for layer 2 should have shape `(output_size, hidden_size)`

Define your initialize_parameters function here

Step 3: Define Activation Functions

Task:

Implement the following activation functions:

- ReLU for the hidden layer
- Sigmoid for the output layer

Also implement the derivative of ReLU for use in backpropagation.

Hint:

- ReLU: `np.maximum(0, Z)`
- Sigmoid: `1 / (1 + np.exp(-Z))`

Define sigmoid, relu, and relu_derivative here

Step 4: Implement Forward Propagation

Task:

Given the input **X** and parameters **W1**, **b1**, **W2**, **b2**, implement the forward pass.

- Compute **Z1**, **A1** (ReLU)
- Compute **Z2**, **A2** (Sigmoid)

Hint:

Store intermediate values in a dictionary called **cache** for use in backpropagation.

Define the forward_propagation function here

Step 5: Compute the Cost Function

Task:

Implement the binary cross-entropy loss function.

Hint:

Use $-Y * \log(A2) - (1 - Y) * \log(1 - A2)$ and take the mean over all examples.

Use **np.squeeze()** to return a scalar.

Define the compute_cost function here

Step 6: Backward Propagation

Task:

Implement the backpropagation algorithm to compute gradients.

- Use the cache from forward pass
- Calculate **dZ2**, **dW2**, **db2**, then backpropagate to get **dZ1**, **dW1**, **db1**

Hint:

Use `relu_derivative` for backpropagating through the hidden layer.

Define the backward_propagation function here

Step 7: Update Parameters

Task:

Write a function to update parameters using gradient descent.

Hint:

Update: `param = param - learning_rate * gradient`

Define the update_parameters function here

Step 8: Train the Neural Network

Task:

Create a function that runs the training loop:

- Repeatedly perform forward and backward propagation
- Update parameters
- Print cost every 100 iterations

Hint:

Use the functions you've created so far.

Define the train_neural_network function here

Step 9: Make Predictions

Task:

Create a function that uses the trained model to make predictions.

Hint:

Use threshold **0.5** to classify outputs.

Define the predict function here

Step 10: Test Your ANN on AND Logic Gate

Task:

Create a small dataset representing the AND gate logic. Train your model and display the predictions.

Hint:

- Input **X** will have shape (2, 4)
- Labels **Y** will have shape (1, 4)

Define X, Y here

Train and test your ANN using the defined functions

Bonus (Optional): Try Other Logic Gates

- Try the same code structure for OR, XOR gates
- What changes in predictions for XOR?

Reference : [ANN from scratch](#)

Lab Task 2: Recreate and Understand Kaggle ANN Classification Tutorial

Objective:

In this lab, interns will re-implement the Kaggle notebook titled:

"Deep Tutorial 1: ANN and Classification" by [@shrutimechlearn](#)

This task aims to help interns understand the complete flow of building, training, and evaluating an Artificial Neural Network (ANN) for classification. Unlike Task 1 (where the ANN was built purely from scratch), here interns are allowed to use external libraries used in the original notebook (such as `pandas`, `matplotlib`, `scikit-learn`, etc.), but they must write and explain every line themselves.

Allowed Tools

- `pandas`
- `numpy`
- `matplotlib` / `seaborn` (for visualization)
- `scikit-learn` (for preprocessing, model selection, and metrics)
- `keras` (optional but recommended to be replaced with manual implementation where possible)

Note: interns should avoid copying code blindly. Instead, they must recode each block and add inline comments to explain the following:

- What each step is doing
 - Why it is necessary
 - What are the expected inputs/outputs
-

Tasks Breakdown

- ♦ Step 1: Load & Understand Dataset
 - Load the dataset used in the Kaggle tutorial (Telecom Customer Churn dataset)

- **Perform Exploratory Data Analysis (EDA)**
- **Handle missing values if any**
- **Visualize correlations and distributions**

◆ **Step 2: Data Preprocessing**

- **Encode categorical variables using Label Encoding or One-Hot Encoding**
- **Normalize/standardize features**
- **Split dataset into training and testing sets**

◆ **Step 3: Build ANN Model**

- **Define ANN architecture using Keras Sequential API (or write equivalent custom logic)**
- **Choose appropriate number of input nodes (equal to number of features)**
- **Use one or more hidden layers with suitable activation functions (e.g. ReLU)**
- **Use a final output layer with sigmoid activation for binary classification**

◆ **Step 4: Compile & Train Model**

- **Compile the model using appropriate loss function (e.g. `binary_crossentropy`), optimizer (e.g. `adam`), and metrics (e.g. `accuracy`)**
- **Train the model on training data**
- **Plot training accuracy/loss vs epochs**

◆ **Step 5: Evaluate Model**

- **Evaluate model performance on test data**

- Plot confusion matrix
 - Calculate precision, recall, F1-score, and ROC AUC score
 - Visualize results using suitable plots (e.g. bar charts, ROC curves)
-

Bonus Challenge (Optional)

- Try increasing the number of layers or neurons
 - Change activation functions and compare performance
 - Add dropout layers for regularization
 - Compare results from **Keras** model vs custom NumPy-only model (Task 1)
-

Submission Guidelines

- Submit a Jupyter Notebook with:
 1. All code blocks well-commented
 2. Visualizations inline
 3. Explanations for every major step
 - Submit a short reflection answering:
 1. What was the hardest part to understand?
 2. How does this ANN differ from the one you built in Task 1?
 3. What would you change if you had more time?
-

Learning Outcome

- Understand practical dataset preprocessing techniques
 - Apply ANN on real-world classification problem
 - Visualize model performance and evaluate results using multiple metrics
 - Build confidence transitioning from low-level to high-level deep learning libraries
 - You now understand how an artificial neural network functions internally, how data propagates forward, how errors are computed and propagated backward, and how learning occurs through gradient descent.
-